Method 1: head

```
[5]: # Display the first few rows
print(df.head())
```

```
         id diagnosis  radius_mean  texture_mean  perimeter_mean  area_mean  \
0    842302         M        17.99         10.38          122.80     1001.0
1    842517         M        20.57         17.77          132.90     1326.0
2  84300903         M        19.69         21.25          130.00     1203.0
3  84348301         M        11.42         20.38           77.58      386.1
4  84358402         M        20.29         14.34          135.10     1297.0

   smoothness_mean  compactness_mean  concavity_mean  concave points_mean  \
0          0.11840           0.27760          0.3001              0.14710
1          0.08474           0.07864          0.0869              0.07017
2          0.10960           0.15990          0.1974              0.12790
3          0.14250           0.28390          0.2414              0.10520
4          0.10030           0.13280          0.1980              0.10430

   ...  radius_worst  texture_worst  perimeter_worst  area_worst  \
0  ...         25.38          17.33           184.60      2019.0
1  ...         24.99          23.41           158.80      1956.0
2  ...         23.57          25.53           152.50      1709.0
```

Method 2: info

```
[6]: # Display dataset info
print(df.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 32 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   id                       569 non-null    int64
 1   diagnosis                569 non-null    object
 2   radius_mean              569 non-null    float64
 3   texture_mean             569 non-null    float64
 4   perimeter_mean           569 non-null    float64
 5   area_mean                569 non-null    float64
 6   smoothness_mean          569 non-null    float64
 7   compactness_mean         569 non-null    float64
 8   concavity_mean           569 non-null    float64
 9   concave points_mean      569 non-null    float64
 10  symmetry_mean            569 non-null    float64
 11  fractal_dimension_mean   569 non-null    float64
 12  radius_se                569 non-null    float64
 13  texture_se               569 non-null    float64
 14  perimeter_se             569 non-null    float64
 15  area_se                  569 non-null    float64
 16  smoothness_se            569 non-null    float64
 17  compactness_se           569 non-null    float64
 18  concavity_se             569 non-null    float64
 19  concave points_se        569 non-null    float64
 20  symmetry_se              569 non-null    float64
 21  fractal_dimension_se     569 non-null    float64
 22  radius_worst             569 non-null    float64
 23  texture_worst            569 non-null    float64
 24  perimeter_worst          569 non-null    float64
 25  area_worst               569 non-null    float64
 26  smoothness_worst         569 non-null    float64
 27  compactness_worst        569 non-null    float64
 28  concavity_worst          569 non-null    float64
 29  concave points_worst     569 non-null    float64
 30  symmetry_worst           569 non-null    float64
 31  fractal_dimension_worst  569 non-null    float64
dtypes: float64(30), int64(1), object(1)
memory usage: 142.4+ KB
None
```

Method 3: value_counts()

```
[14]: # Apply value_counts()
print(df['diagnosis'].value_counts())
```

```
diagnosis
B    357
M    212
Name: count, dtype: int64
```

Method 4: describe

```
[7]: # Display summary statistics of numerical attributes
df.describe()
```

[7]:

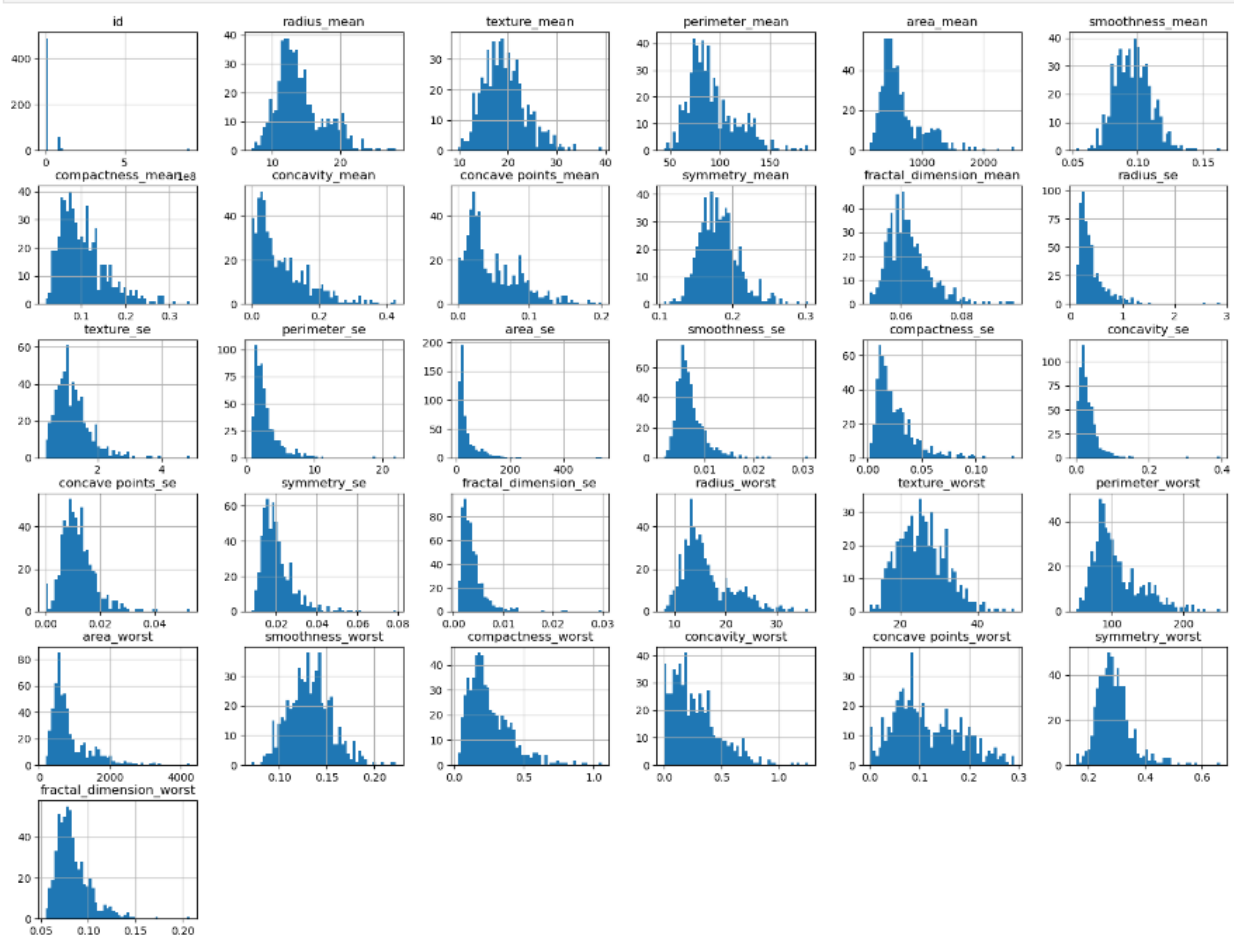
	id	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	concavity_mean	concave points_mean	symmetry_mean
count	5.690000e+02	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000
mean	3.037183e+07	14.127292	19.289649	91.969033	654.889104	0.096360	0.104341	0.088799	0.048919	0.181162
std	1.250206e+08	3.524049	4.301036	24.298981	351.914129	0.014064	0.052813	0.079720	0.038803	0.027414
min	8.670000e+03	6.981000	9.710000	43.790000	143.500000	0.052630	0.019380	0.000000	0.000000	0.106000
25%	8.692180e+05	11.700000	16.170000	75.170000	420.300000	0.086370	0.064920	0.029560	0.020310	0.161900
50%	9.060240e+05	13.370000	18.840000	86.240000	551.100000	0.095870	0.092630	0.061540	0.033500	0.179200
75%	8.813129e+06	15.780000	21.800000	104.100000	782.700000	0.105300	0.130400	0.130700	0.074000	0.195700
max	9.113205e+08	28.110000	39.280000	188.500000	2501.000000	0.163400	0.345400	0.426800	0.201200	0.304000

8 rows × 31 columns

Method 5: hist

```
# Plot histograms for all numerical columns
df.hist(bins=50, figsize=(20, 15))

# Show the plot
plt.show()
```



Findings

Method 1: The head() method displayed the first five rows of the dataset, giving a glimpse of its structure. This includes the "id," "diagnosis," and various other attributes such as radius, texture, perimeter, area, smoothness, and compactness, each with mean, standard error, and worst-case values. The "diagnosis" column identifies whether the tumor is benign (B) or malignant (M), while the remaining columns consist of numerical values that represent tumor characteristics. This preview provides insight into the types of data in the dataset, including both numerical and categorical attributes.

Method 2: The info() method offered an overview of the dataset, confirming it contains 569 entries and 32 columns. Among these, 30 columns are of type float64, with one column (id) of type int64. The "diagnosis" column is categorical, represented as an object. The method also confirmed that there are no missing values, suggesting that the dataset is complete and ready for further analysis. This summary highlights the types of data and the overall integrity of thedataset.

Method 3: The value_counts() method applied to the "diagnosis" column showed that the dataset contains 357 benign (B) cases and 212 malignant (M) cases. This indicates an imbalance, with more benign than malignant tumors. Such an imbalance is important to note for machine learning models, as it might require techniques such as resampling or class weighting to avoid bias towards predicting benign tumors.

Method 4: The describe() method provided a statistical summary of the numerical columns, including key metrics such as mean, standard deviation, minimum, and maximum values. For example, the mean value for "radius_mean" is 14.13, while "area_mean" averages 654.89. The data reveals a broad range, with some columns like "area_worst" reaching values up to 4,254, highlighting the diversity in tumor size and characteristics. This method offers insight into the central tendency and spread of the numerical data, informing future data processing and feature selection.

Method 5: The histogram analysis of various tumor attributes showed that most features, such as radius and texture, exhibit right-skewed distributions, especially in the standard error measurements. Many features show clear peaks near their central values, with the distribution tapering off in either direction. Notably, area measurements extend across a wide range, while features like smoothness and symmetry cluster within narrower ranges. The mean measurements tend to follow more normal-like distributions, whereas the standard error and worst-case values show more pronounced right skewness. These characteristics, indicative of various tumor attributes (e.g., radius, texture, concavity), suggest the dataset is likely related to medical imaging, such as cell or tissue analysis.